

CHULETA DE JAVASCRIPT

Funciones, funciones flecha y métodos de Arrays

Esta chuleta está pensada para estudiar los conceptos utilizados en el ejercicio de la tienda de música. La idea principal es entender qué significa cada parte del código y saber reconocer qué método debemos utilizar dependiendo de lo que nos pida el ejercicio.

1. ¿Qué es una función?

Una función es un bloque de código que sirve para realizar una tarea concreta. Una función puede recibir información, realizar una operación con ella y devolver un resultado. Por ejemplo, podemos crear una función llamada `sumar` que reciba dos números y devuelva su suma.

```
function sumar(a, b) {  
    return a + b;  
}
```

En este caso, `a` y `b` son los parámetros de la función. Si hacemos `sumar(3, 5)`, la función recibe los valores 3 y 5 y devuelve 8. Podemos imaginar una función como una máquina: le damos unos datos, la función los procesa y nos devuelve un resultado.

2. ¿Qué es una función flecha?

Una función flecha es simplemente otra forma de escribir una función en JavaScript. La forma tradicional sería la siguiente:

```
function sumar(a, b) {  
    return a + b;  
}
```

La misma función utilizando una función flecha se puede escribir así:

```
const sumar = (a, b) => {  
    return a + b;  
}
```

La parte `=>` es lo que identifica a una función flecha. Podemos leer `const sumar = (a, b) =>` como "creo una función llamada `sumar` que recibe `a` y `b`".

En tu ejercicio vas a encontrar constantemente este tipo de funciones. Por ejemplo:

```
const obtenerNombre = (producto) => {  
    return producto.nombre;  
}
```

```
}
```

Esto significa: "creo una función que recibe un producto y devuelve su nombre".

3. Funciones flecha abreviadas

Las funciones flecha se pueden escribir de una forma todavía más corta cuando solamente necesitamos devolver un resultado. Por ejemplo, esta función:

```
const obtenerNombre = (producto) => {  
    return producto.nombre;  
}
```

se puede escribir:

```
const obtenerNombre = producto => producto.nombre;
```

Las dos formas hacen exactamente lo mismo. Cuando escribimos una función flecha sin llaves y ponemos directamente una expresión después de `=>`, JavaScript entiende que queremos devolver ese resultado automáticamente.

Por eso, cuando veas algo como `producto => producto.nombre`, puedes leerlo mentalmente como "recibe un producto y devuelve su nombre". Del mismo modo, `numero => numero * 2` significa "recibe un número y devuelve ese número multiplicado por dos".

4. ¿Qué significa const?

`const` sirve para crear una variable cuyo valor no se puede reasignar posteriormente. En los ejercicios se utiliza mucho para guardar funciones.

Por ejemplo:

```
const sumar = (a, b) => {  
    return a + b;  
}
```

Aquí estamos creando una constante llamada `sumar` y guardando dentro de ella una función.

En tu ejercicio aparece:

```
export const crearCatalogo = (matriz) => {
```

Esto significa que estamos creando una función llamada `crearCatalogo` que recibe un parámetro llamado `matriz`.

5. ¿Qué significa `export`?

La palabra `export` permite que una función pueda utilizarse desde otro archivo. En tu ejercicio, las funciones están dentro de `tienda.js`, mientras que otros archivos como `pruebas.js` pueden necesitar utilizarlas. Por eso aparecen declaraciones como:

```
export const crearCatalogo = (matriz) => {  
  // código  
}
```

Para el examen puedes recordar simplemente que `export` permite exportar una función para utilizarla desde otro archivo.

6. Parámetros

Los parámetros son los datos que recibe una función. Por ejemplo, en la siguiente función `nombre` es un parámetro:

```
const saludar = (nombre) => {  
  return "Hola " + nombre;  
}
```

Si hacemos `saludar("Lucía")`, la función recibe "Lucía" como valor de `nombre` y devuelve "Hola Lucía".

En tu ejercicio podemos tener varios parámetros. Por ejemplo:

```
export const buscarProducto = (catalogo, nombre) => {  
  // código  
}
```

Esta función recibe dos datos: un `catalogo` y un `nombre`.

7. `return`

`return` sirve para devolver un resultado desde una función. Por ejemplo:

```
const doble = (numero) => {  
  return numero * 2;  
}
```

Si llamamos a `doble(5)`, la función devuelve 10. Es importante recordar que `return` termina la ejecución de esa función y devuelve el resultado indicado.

8. if

if significa "si" y sirve para ejecutar un bloque de código solamente cuando se cumple una condición. Por ejemplo:

```
if (producto.stock === 0) {  
    return "Agotado";  
}
```

Esto se puede leer como: "si el stock del producto es igual a cero, devuelve 'Agotado'".

También podemos utilizar else, que significa "si no":

```
if (edad >= 18) {  
    return "Adulto";  
} else {  
    return "Menor";  
}
```

9. Operadores importantes

El operador === significa "es exactamente igual a". Por ejemplo, producto.nombre === nombre comprueba si el nombre del producto es igual al nombre que estamos buscando.

El operador !== significa "es diferente de". Por ejemplo, producto !== undefined significa que el producto existe y no es undefined.

Los operadores >= y <= significan "mayor o igual que" y "menor o igual que". Son muy utilizados para comprobar rangos de precios o cantidades.

El operador && significa "Y". Cuando tenemos:

```
producto.precio >= minimo &&  
producto.precio <= maximo
```

las dos condiciones tienen que cumplirse. Es decir, el precio tiene que ser mayor o igual que el mínimo y, además, menor o igual que el máximo.

El operador || significa "O". En este caso solo es necesario que se cumpla una de las condiciones.

10. Arrays

Un array es una lista de elementos. Por ejemplo:

```
const numeros = [10, 20, 30, 40];
```

Los elementos de un array tienen posiciones y estas empiezan en 0. Por tanto, `numeros[0]` devuelve 10, `numeros[1]` devuelve 20 y así sucesivamente.

Los arrays son fundamentales en tu ejercicio porque el catálogo, los pedidos y el carrito están representados mediante arrays.

11. Objetos

Un objeto permite guardar diferentes datos relacionados utilizando propiedades. Por ejemplo, un producto puede representarse así:

```
const producto = {  
  nombre: "Tocadiscos",  
  categoria: "equipos",  
  precio: 200,  
  stock: 5  
};
```

Para acceder al nombre utilizamos `producto.nombre`, mientras que para acceder al precio utilizamos `producto.precio`. En tu ejercicio todos los productos tienen propiedades como nombre, categoria, precio y stock.

12. map()

El método `map()` sirve para recorrer todos los elementos de un array y transformarlos. Es decir, recibe cada elemento y genera un nuevo array con el resultado de la transformación.

Por ejemplo:

```
const numeros = [1, 2, 3];  
  
const resultado = numeros.map(numero => numero * 2);
```

El resultado será `[2, 4, 6]`. La función `numero => numero * 2` significa "recibe un número y devuelve ese número multiplicado por dos".

En tu ejercicio podemos utilizar `map()` para obtener los nombres de todos los productos:

```
catalogo.map(producto => producto.nombre);
```

Esto significa "recorre todos los productos y devuelve el nombre de cada uno".

La idea que debes recordar es: **MAP = transformar todos los elementos**.

13. filter()

El método `filter()` sirve para seleccionar solamente los elementos que cumplen una determinada condición. A diferencia de `map()`, no transforma necesariamente los elementos, sino que decide cuáles se quedan y cuáles se descartan.

Por ejemplo:

```
const numeros = [1, 2, 3, 4];
```

```
const resultado = numeros.filter(numero => numero > 2);
```

El resultado será `[3, 4]`, porque solamente esos números cumplen la condición `numero > 2`.

En tu ejercicio:

```
catalogo.filter(producto => producto.stock === 0);
```

significa "quédate solamente con los productos cuyo stock sea igual a cero".

La idea que debes recordar es: **FILTER = filtrar o seleccionar elementos**.

14. Diferencia entre map() y filter()

Esta diferencia es especialmente importante. `map()` se utiliza cuando queremos transformar todos los elementos, mientras que `filter()` se utiliza cuando queremos quedarnos solamente con algunos.

Por ejemplo, si tenemos `[1, 2, 3]`, hacer `map(numero => numero * 2)` produce `[2, 4, 6]`, porque hemos transformado todos los elementos. En cambio, `filter(numero => numero > 1)` produce `[2, 3]`, porque hemos seleccionado únicamente los elementos que cumplen la condición.

Una forma sencilla de recordarlo es: **map cambia; filter selecciona**.

15. find()

El método `find()` sirve para buscar un elemento dentro de un array. Devuelve el primer elemento que cumple la condición indicada.

En tu ejercicio podemos hacer:

```
const producto = catalogo.find(
  producto => producto.nombre === "Tocadiscos"
);
```

JavaScript recorre los productos hasta encontrar uno cuyo nombre sea "Tocadiscos". Si lo encuentra, devuelve el objeto completo. Si no encuentra ninguno, devuelve `undefined`.

Por tanto, `find()` significa: **buscar un elemento**.

16. `findIndex()`

`findIndex()` funciona de manera parecida a `find()`, pero en lugar de devolver el objeto devuelve su posición.

Por ejemplo:

```
const posicion = catalogo.findIndex(
  producto => producto.nombre === "Tocadiscos"
);
```

Si el producto está en la posición 2, devuelve 2. Si no existe, devuelve -1.

La diferencia fundamental es: `find()` devuelve el objeto, mientras que `findIndex()` devuelve su posición.

17. `includes()`

`includes()` sirve para comprobar si un array contiene un determinado valor. Devuelve `true` si existe y `false` si no existe.

Por ejemplo:

```
const nombres = ["Tocadiscos", "Vinilo", "Guitarra"];

nombres.includes("Vinilo");
```

El resultado será `true`.

En tu ejercicio se utiliza junto con `map()`:

```
catalogo
  .map(producto => producto.nombre)
  .includes(nombre);
```

Primero obtenemos un array con los nombres y después comprobamos si el nombre buscado está incluido.

La idea que debes recordar es: **INCLUDES = comprobar si existe.**

18. some()

`some()` sirve para comprobar si al menos un elemento cumple una condición. Devuelve `true` si encuentra alguno y `false` si ninguno cumple.

Por ejemplo:

```
const numeros = [1, 2, 3, 4];

numeros.some(numero => numero > 3);
```

El resultado es `true` porque existe un número mayor que 3.

En tu ejercicio:

```
catalogo.some(producto => producto.stock === 0);
```

significa "¿hay al menos un producto agotado?"

La palabra clave para recordar `some()` es: **ALGUNO.**

19. every()

`every()` sirve para comprobar si todos los elementos cumplen una condición. Devuelve `true` solamente si todos cumplen la condición.

Por ejemplo:

```
const numeros = [2, 4, 6];

numeros.every(numero => numero % 2 === 0);
```

El resultado es `true` porque todos los números son pares.

En tu ejercicio:

```
catalogo.every(
  producto => typeof producto.precio === "number" &&
    producto.precio > 0
```

);

significa "¿todos los productos tienen un precio que sea un número y además mayor que cero?"

La palabra clave para recordar `every()` es: **TODOS**.

20. `reduce()`

`reduce()` es uno de los métodos más importantes del ejercicio. Sirve para recorrer un array y acumular información hasta obtener un único resultado. Es muy habitual utilizarlo para sumar valores.

Por ejemplo:

```
const numeros = [10, 20, 30];

const total = numeros.reduce((total, numero) => {
  return total + numero;
}, 0);
```

Primero empezamos con `total = 0`. Después sumamos 10 y obtenemos 10. Después sumamos 20 y obtenemos 30. Finalmente sumamos 30 y obtenemos 60.

La estructura habitual es:

```
array.reduce((acumulador, elemento) => {
  return acumulador + algo;
}, valorInicial);
```

La idea que debes recordar es: **REDUCE = acumular o conseguir un resultado final**.

21. `reduce()` para calcular el valor del almacén

En tu ejercicio aparece:

```
return catalogo.reduce((total, producto) => {
  return total + producto.precio * producto.stock;
}, 0);
```

Para cada producto calculamos `precio × stock` y añadimos ese resultado al total acumulado.

Por ejemplo, si tenemos 5 tocadiscos a 200 €, tendremos $200 \times 5 = 1000$ €. Si además tenemos 10 vinilos a 30 €, tendremos $30 \times 10 = 300$ €. El valor total será 1300 €.

22. sort()

`sort()` sirve para ordenar un array. Para ordenar números de menor a mayor podemos utilizar:

```
array.sort((a, b) => a - b);
```

Para ordenar números de mayor a menor:

```
array.sort((a, b) => b - a);
```

Para ordenar textos alfabéticamente podemos utilizar:

```
array.sort((a, b) => a.localeCompare(b));
```

En tu ejercicio aparece `localeCompare()` porque permite ordenar correctamente textos y respetar las características de diferentes idiomas, como las tildes.

Hay que tener cuidado porque `sort()` modifica el array original. Si no queremos modificarlo, podemos hacer primero una copia:

```
const copia = [...array];
```

```
copia.sort((a, b) => a - b);
```

La idea que debes recordar es: **SORT = ordenar**.

23. Spread operator (...)

Los tres puntos `...` se conocen como operador spread. En tu ejercicio se utilizan principalmente para copiar arrays y objetos.

Para copiar un array podemos escribir:

```
const copia = [...catalogo];
```

Para copiar un objeto podemos escribir:

```
const copia = {...producto};
```

También podemos utilizarlo para copiar un objeto y modificar solamente una propiedad:

```
const nuevoProducto = {  
  ...producto,  
  stock: 10  
};
```

Esto significa "copia todas las propiedades de producto, pero establece el stock en 10".

24. slice()

slice() sirve para obtener una parte de un array sin modificar el original.

Por ejemplo:

```
const numeros = [10, 20, 30, 40];  
  
const resultado = numeros.slice(0, 2);
```

El resultado será [10, 20]. El primer número indica dónde empezamos y el segundo indica dónde terminamos, pero esa posición final no se incluye.

Por eso:

```
array.slice(0, 3);
```

significa "coge los elementos de las posiciones 0, 1 y 2".

En tu ejercicio:

```
ordenarPorPrecio(catalogo)  
  .slice(0, 3)
```

significa "ordena el catálogo por precio y quédate con los tres primeros productos".

25. split()

split() sirve para separar un texto utilizando un determinado carácter.

Por ejemplo:

```
const texto = "Hola-Mundo";
```

```
const resultado = texto.split("-");
```

El resultado será:

```
[  
  "Hola",  
  "Mundo"  
]
```

En tu ejercicio se utiliza para separar el cliente de las líneas del pedido:

```
const partes = texto.split("|");
```

Si tenemos "Lucía|Tocadiscos:1;Vinilo Jazz:2", obtenemos dos partes: "Lucía" y "Tocadiscos:1;Vinilo Jazz:2".

Después podemos utilizar otro `split()` para separar los diferentes productos utilizando `;`, y otro para separar el nombre de la cantidad utilizando `:`.

La idea que debes recordar es: **SPLIT = separar texto**.

26. Number()

`Number()` sirve para convertir un valor en un número.

Por ejemplo:

```
const cantidad = "2";
```

```
const numero = Number(cantidad);
```

Antes, "2" era un texto. Después de utilizar `Number()`, tenemos el número 2.

Esto es importante en el ejercicio porque las cantidades del pedido vienen inicialmente como texto.

27. join()

`join()` sirve para unir los elementos de un array formando un texto.

Por ejemplo:

```
const palabras = ["Hola", "qué", "tal"];
```

```
const resultado = palabras.join(" ");
```

El resultado será "Hola qué tal".

En el ticket utilizamos:

```
lineas.join("\n");
```

\n representa un salto de línea, por lo que conseguimos que cada parte del ticket aparezca en una línea diferente.

La idea que debes recordar es: **JOIN = unir**.

28. push()

push() añade un elemento al final de un array.

```
const carrito = ["Vinilo"];
```

```
carrito.push("Tocadiscos");
```

El resultado será:

```
[  
  "Vinilo",  
  "Tocadiscos"  
]
```

La palabra clave es: **PUSH = añadir al final**.

29. pop()

pop() elimina el último elemento de un array.

```
const carrito = ["Vinilo", "CD", "Tocadiscos"];
```

```
carrito.pop();
```

Después tendremos:

```
[  
  "Vinilo",  
  "CD"  
]
```

La palabra clave es: **POP = quitar del final.**

30. unshift()

unshift() añade un elemento al principio del array.

```
const cola = ["Pedido 2", "Pedido 3"];
```

```
cola.unshift("Pedido 1");
```

Ahora tendremos:

```
[  
  "Pedido 1",  
  "Pedido 2",  
  "Pedido 3"  
]
```

La palabra clave es: **UNSHIFT = añadir al principio.**

31. shift()

shift() elimina el primer elemento del array.

```
const cola = ["Pedido 1", "Pedido 2", "Pedido 3"];
```

```
cola.shift();
```

Después tendremos:

```
[  
  "Pedido 2",  
  "Pedido 3"  
]
```

La palabra clave es: **SHIFT = quitar del principio.**

32. Tabla para recordar push, pop, shift y unshift

La forma más sencilla de memorizar estos cuatro métodos es pensar en el principio y el final del array.

unshift() añade al principio, shift() quita del principio, push() añade al final y pop() quita del final.

En forma de tabla mental:

	PRINCIPIO	FINAL
AÑADIR	<code>unshift()</code>	<code>push()</code>
QUITAR	<code>shift()</code>	<code>pop()</code>

33. `indexOf()`

`indexOf()` sirve para buscar la posición de un elemento. Si el elemento existe, devuelve su posición. Si no existe, devuelve `-1`.

Por ejemplo:

```
const carrito = ["CD", "Vinilo", "Guitarra"];

carrito.indexOf("Vinilo");
```

El resultado será `1`, porque "Vinilo" está en la posición `1`.

La idea que debes recordar es: **INDEXOF = buscar posición**.

34. `lastIndexOf()`

`lastIndexOf()` funciona como `indexOf()`, pero busca la última aparición de un elemento.

Por ejemplo:

```
const carrito = [
  "Vinilo",
  "CD",
  "Vinilo"
];

carrito.lastIndexOf("Vinilo");
```

El resultado será `2`.

Por tanto, `indexOf()` busca la primera aparición y `lastIndexOf()` busca la última.

35. `splice()`

`splice()` sirve para eliminar o insertar elementos en una posición determinada.

Para eliminar un elemento podemos hacer:

```
array.splice(posicion, 1);
```

El segundo número indica cuántos elementos queremos eliminar.

Para insertar un elemento podemos utilizar:

```
array.splice(posicion, 0, elemento);
```

El 0 significa que no queremos eliminar ningún elemento, solamente insertar.

Por tanto, para memorizarlo:

```
splice(posicion, 1);
```

significa eliminar.

```
splice(posicion, 0, elemento);
```

significa insertar.

36. while()

while significa "mientras". Sirve para repetir un bloque de código mientras se cumpla una condición.

En tu ejercicio aparece:

```
while (cola.length > 0) {  
    const pedido = atenderSiguiente(cola);  
  
    // procesar pedido  
}
```

Esto significa "mientras la cola tenga pedidos, sigue procesando pedidos".

Cuando la cola quede vacía, `cola.length > 0` será falso y el bucle terminará.

La palabra clave es: **WHILE = mientras**.

37. Set

Set permite trabajar con valores sin repetir. Es muy útil cuando el ejercicio dice que queremos un resultado "sin repetidos".

Por ejemplo:

```
const nombres = [  
  "Vinilo",  
  "CD",  
  "Vinilo",  
  "Guitarra"  
];  
  
const resultado = [...new Set(nombres)];
```

El resultado será:

```
[  
  "Vinilo",  
  "CD",  
  "Guitarra"  
]
```

El patrón que debes memorizar es:

```
[...new Set(array)]
```

Esto significa: "crear un conjunto para eliminar los repetidos y después volver a convertirlo en un array".

38. flatMap()

`flatMap()` permite recorrer elementos y juntar los resultados. En tu ejercicio es útil porque cada pedido contiene varias líneas y queremos obtener todos los nombres de productos vendidos.

Por ejemplo:

```
pedidos.flatMap(  
  pedido => pedido.lineas.map(  
    linea => linea.nombre  
  )  
);
```

Primero entramos en cada pedido, después recorremos sus líneas y obtenemos el nombre de cada línea. `flatMap()` se encarga de juntar todos los resultados en un único array.

39. `new Array().fill()`

En el ejercicio tenemos que crear una barra visual para representar el stock. Para ello utilizamos `new Array()` junto con `fill()`.

Por ejemplo:

```
new Array(3).fill("■");
```

crea tres posiciones que contienen el símbolo ■.

Después podemos utilizar `join("")` para convertirlo en un texto:

```
new Array(3).fill("■").join("");
```

Resultado:

■■■

En el ejercicio, el número de símbolos depende del stock:

```
new Array(producto.stock)
  .fill("■")
  .join("");
```

Por tanto, si el stock es 5, tendremos cinco cuadrados.

40. Encadenar métodos

En JavaScript es muy habitual utilizar varios métodos uno detrás de otro.

Por ejemplo:

```
catalogo
  .filter(producto => producto.stock === 0)
  .map(producto => producto.nombre);
```

Primero se ejecuta `filter()`, que selecciona los productos agotados. Después se ejecuta `map()`, que obtiene únicamente el nombre de esos productos.

Podemos imaginarlo como una cadena:

CATÁLOGO → FILTER → PRODUCTOS AGOTADOS → MAP → NOMBRES

Esta forma de trabajar aparece muchas veces en tu ejercicio.

41. Cómo interpretar una función flecha

Esta es probablemente la parte más importante para entender el código.

Cuando veas:

```
producto => producto.nombre
```

léelo como:

"Dado un producto, devuelve su nombre."

Cuando veas:

```
numero => numero * 2
```

léelo como:

"Dado un número, devuelve el número multiplicado por dos."

Cuando veas:

```
producto => producto.stock === 0
```

léelo como:

"Dado un producto, comprueba si su stock es igual a cero."

Una función flecha normalmente tiene esta estructura:

```
parametro => resultado
```

Por eso puedes pensar:

```
producto → entra un producto  
=>          → hacemos una operación  
producto.nombre → resultado
```

42. Ejemplo completo de tu ejercicio

Esta función puede parecer complicada al principio:

```
export const agotados = (catalogo) => {  
  return catalogo  
    .filter(producto => producto.stock === 0)  
    .map(producto => producto.nombre);  
}
```

```
};
```

Pero si la dividimos en partes es muy sencilla. Primero estamos creando una función llamada `agotados` que recibe un `catalogo`. Después utilizamos `filter()` para quedarnos únicamente con los productos cuyo `stock` sea igual a cero. Finalmente utilizamos `map()` para convertir esos productos en sus nombres. Por último, `return` devuelve el resultado.

Por ejemplo, si el catálogo contiene un `Tocadiscos` con `stock` 5 y un `Vinilo` con `stock` 0, el resultado será:

```
[  
  "Vinilo"  
]
```

43. Tabla definitiva para el examen

Cuando el enunciado diga "transformar todos los elementos", piensa en `map()`. Cuando diga "quedarse con los que cumplen una condición", piensa en `filter()`. Cuando diga "buscar un producto", piensa en `find()`. Si pide la posición, utiliza `findIndex()`. Si pregunta si existe un elemento, utiliza `includes()`. Si pregunta si hay al menos uno que cumple una condición, utiliza `some()`. Si pregunta si todos cumplen una condición, utiliza `every()`. Si pide una suma, un total o acumular información, normalmente utilizaremos `reduce()`. Si pide ordenar, utilizaremos `sort()`. Si pide coger una parte del array, podemos utilizar `slice()`. Para separar un texto utilizamos `split()` y para unir elementos utilizamos `join()`.

En la parte de modificación de arrays, `push()` añade al final, `pop()` elimina del final, `unshift()` añade al principio y `shift()` elimina del principio. `splice()` sirve para eliminar o insertar elementos en una posición concreta. `indexOf()` busca la primera posición de un elemento y `lastIndexOf()` busca la última.

Si el ejercicio pide eliminar elementos repetidos, podemos utilizar `Set`. Si necesitamos copiar un array utilizamos `[...array]` y si queremos copiar un objeto utilizamos `{...objeto}`.

44. Las palabras clave que debes memorizar

La forma más rápida de estudiar estos métodos es asociar cada uno con una palabra:

`map()` significa TRANSFORMAR.

`filter()` significa FILTRAR.

`find()` significa BUSCAR.

`findIndex()` significa POSICIÓN.

`includes()` significa EXISTE.

`some()` significa ALGUNO.

`every()` significa TODOS.

`reduce()` significa ACUMULAR.

`sort()` significa ORDENAR.

`slice()` significa CORTAR O COGER UNA PARTE.

`split()` significa SEPARAR.

`join()` significa UNIR.

`push()` significa AÑADIR AL FINAL.

`pop()` significa QUITAR DEL FINAL.

`unshift()` significa AÑADIR AL PRINCIPIO.

`shift()` significa QUITAR DEL PRINCIPIO.

`splice()` significa ELIMINAR O INSERTAR.

`Set` significa ELIMINAR REPETIDOS.

`while` significa REPETIR MIENTRAS.

45. Regla final para reconocer las funciones flecha

Siempre que veas algo parecido a:

`x => algo`

puedes leerlo como:

"Dado x, devuelve algo."

Por ejemplo:

`producto => producto.nombre`

significa "dado un producto, devuelve su nombre".

```
numero => numero * 2
```

significa "dado un número, devuelve el doble".

```
producto => producto.stock === 0
```

significa "dado un producto, comprueba si está agotado".

Una vez que entiendas esta estructura, métodos como `map()`, `filter()`, `find()`, `some()` y `every()` serán mucho más fáciles de entender.

RESUMEN FINAL

La idea principal es no intentar memorizar el código completo de cada función, sino entender qué herramienta necesitas según lo que te pida el enunciado. Si necesitas transformar elementos, piensa en `map()`. Si necesitas seleccionar algunos, piensa en `filter()`. Si necesitas buscar uno, utiliza `find()`. Si necesitas comprobar una condición en todos, utiliza `every()`, y si solamente necesitas saber si existe alguno que la cumpla, utiliza `some()`. Para sumar o acumular utiliza `reduce()`. Para ordenar utiliza `sort()`. Para modificar posiciones concretas utiliza `splice()`.

Y recuerda especialmente que una función flecha como `producto => producto.nombre` simplemente significa: "recibe un producto y devuelve su nombre".